

AuraGreet Data Journey

A comprehensive end-to-end technical overview detailing data transit, storage execution layers, API proxies, and UI rendering mechanics.

1. Complete Traffic Flow Overview

The sequence below traces the logical path of data transactions during name storage operations:

1

USER → KEYBOARD EVENT

Keystroke Triggers HTML Event

The physical keyboard keys are registered by the browser, firing client-side JavaScript input handlers to update the monitor greeting live.

2

FRONTEND → NETWORK

Form Intercept & API Serialization

Pressing enter locks the input. The Supabase JS Client serializes the string name into a JSON array, signing the package with the Anon JWT bearer token.

3

NETWORK → KONG API GATEWAY

Gateway Authentication & CORS Inspection

The encrypted HTTPS request lands on Kong. The API gateway verifies access permissions, CORS configurations, and token signatures.

4

KONG GATEWAY → POSTGRES ENGINE

REST API Endpoint to SQL Translation

PostgREST translates the JSON headers and REST routes into standard SQL insert statements, maintaining PostgreSQL database transaction logs.

5

POSTGRES → DATABASE (POSTGRESQL)

Row Level Security (RLS) & Storage Commit

PostgreSQL runs the query against the `Allow public read and insert` RLS rule. If allowed, it generates a sequential primary ID and writes the row to the Write-Ahead Log (WAL) on disk.

6

DATABASE → FRONTEND CLIENT

Response Callback & UI Redraw

A success status is returned. The frontend client triggers a refresh query, downloads the latest 10 logs, and redraws the data logger rows on the same screen.

2. Frontend Client Mechanics

The client side acts as the interface, executing capture operations in three main phases:

A. Keystroke Listening

The application registers user input live as they type. Inside `app.js`, an event listener hooks into the input element:

```
userNameInput.addEventListener('input', (e) => {  
  const textValue = e.target.value;  
  if (textValue.trim().length === 0) {  
    displayedName.textContent = "Guest";  
    return;  
  }  
  displayedName.textContent = textValue;  
});
```

B. Serialization

Upon submission, the text is extracted, sanitized, and serialized into JSON. The Supabase SDK structures the data package for transfer:

```
// Client-side payload packaging  
const insertRow = { Name: "John" };  
await supabaseClient.from("Customer Names").insert([insertRow]);
```

C. Dynamic Rendering

After logging names, JavaScript executes a query to read the logs. It formats the server datetime stamps, sanitizes names to prevent script injection attacks (XSS), and dynamically inserts table markup elements.

3. Backend Infrastructure (Supabase API Gateway)

Supabase abstracts the backend layer using a dedicated container architecture that acts as a proxy proxying REST transactions to PostgreSQL:

A. Kong API Gateway

Kong is the first layer of contact for the web application. It acts as a gatekeeper, verifying that incoming HTTPS requests contain a valid project API key. It parses request metadata, confirms CORS origins, and enforces rate limit controls.

B. PostgREST Translation

Because relational databases communicate natively via SQL, web browsers cannot query Postgres directly. PostgREST bridges this gap by automatically converting HTTP requests into SQL commands. For example:

HTTP Method	API URL Endpoint	PostgreSQL SQL Translation
POST	/rest/v1/Customer%20Names	INSERT INTO public."Customer Names" ("Name") VALUES ('NameValue');
GET	/rest/v1/Customer%20Names? limit=10	SELECT * FROM public."Customer Names" LIMIT 10;

4. Cloud Database Engine (PostgreSQL)

At the final stage, data enters the PostgreSQL relational database engine which manages execution, integrity, and safety:

A. Row Level Security (RLS) Filter

Before any insertion happens, PostgreSQL inspects permissions. Because RLS is active on the `Customer Names` table, the database checks the query against your security policy:

```
-- The SQL policy validating public access
CREATE POLICY "Allow public read and insert"
ON "Customer Names"
FOR ALL
TO public
USING (true)
WITH CHECK (true);
```

This allows anonymous public connections (authenticated via the JWT key) to safely execute both insert and read operations.

B. Transaction Execution

Once allowed, the database opens a secure transaction. It appends the operation to the **Write-Ahead Log (WAL)** on disk to guarantee transaction safety (Durability). Finally, it writes the record, populates the primary ID sequence, stamps the current server timestamp via `now()`, and commits the transaction.